

Introduction

Behavioral design patterns focus on how objects interact and communicate with each other, helping to define the flow of control in a system. These patterns simplify complex communication logic between objects while promoting loose coupling.

Imagine a notification system where multiple users get alerts when a new blog post is published. The publisher shouldn't have to worry about who all are subscribed or how they get notified. This kind of automatic, event-driven update mechanism is exactly what behavioral patterns help us achieve.

One such pattern is the **Observer Pattern**. Let's explore the Observer Pattern in depth in the upcoming sections.

Observer Pattern

The **Observer Pattern** is a behavioral design pattern that defines a one-to-many dependency between objects so that when one object (the **subject**) changes its state, all its dependents (called **observers**) are notified and updated automatically.

Formal Definition

The Observer Pattern is a behavioral design pattern where an object, known as the subject, maintains a list of dependents (observers) and notifies them of any state changes, usually by calling one of their methods.

This means if multiple objects are watching another object for updates, they don't need to keep checking repeatedly. Instead, they get notified as soon as something changes — making the system more efficient and loosely coupled.

Real-Life Analogy

Think of subscribing to a YouTube channel. Once you hit the **Subscribe** button and turn on notifications, you don't have to keep visiting the channel to check for new videos. As soon as a new video is uploaded, **you get notified instantly**.

In this case:

- The **channel** is the **subject**.
- The **subscribers** are the **observers**.
- The **notification** is the automatic update mechanism triggered by the subject.

Similarly, in software, when an object (subject) undergoes a change, all registered observers get notified, just like YouTube alerts its subscribers.

Understanding the Problem

Let's say we're building a simple **YouTube-like Notification System**. Whenever a creator uploads a new video, all their subscribers should get notified.

Below is a naive implementation of this logic:

```
#include <bits/stdc++.h>

using namespace std;
```

```
class YouTubeChannel {  
  
public:  
  
    void uploadNewVideo(string videoTitle) {  
  
        // Upload the video  
  
        cout << "Uploading: " << videoTitle << "\n\n";  
  
  
        // Manually notify users  
  
        cout << "Sending email to user1@example.com" << endl;  
  
        cout << "Pushing in-app notification to user3@example.com" << endl;  
  
    }  
  
};  
  
  
int main() {  
  
    // Create a channel and upload a new video  
  
    YouTubeChannel channel;  
  
    channel.uploadNewVideo("Design Patterns in C++");  
  
  
    return 0;  
  
}
```

```
import java.util.*;
```

```
class YouTubeChannel {
```

```
    public void uploadNewVideo(String videoTitle) {
```

```
        // Upload the video
```

```
        System.out.println("Uploading: " + videoTitle + "\n");
```

```
        // Manually notify users
```

```
        System.out.println("Sending email to user1@example.com");
```

```
        System.out.println("Pushing in-app notification to  
user3@example.com");
```

```
    }
```

```
}
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        // Create a channel and upload a new video
```

```
        YouTubeChannel channel = new YouTubeChannel();
```

```
        channel.uploadNewVideo("Design Patterns in Java");
```

```
    }
```

```
}
```

```

class YouTubeChannel:

    def upload_new_video(self, video_title):

        # Upload the video

        print(f"Uploading: {video_title} \n")


        # Manually notify users

        print("Sending email to user1@example.com")

        print("Pushing in-app notification to user3@example.com")


def main():

    # Create a channel and upload a new video

    channel = YouTubeChannel()

    channel.upload_new_video("Design Patterns in Python")


if __name__ == "__main__":

    main()

```

What's Wrong with This Approach?

While the code works, there are several **design-level concerns**:

- **Tightly Coupled Code:**

The `YouTubeChannel1` class is directly responsible for how users are

notified. If tomorrow we want to send an SMS or push notification, we'll have to edit this class.

- **No Reusability:**

The notification logic (email, app, etc.) is hardcoded. You can't reuse or extend this behavior in other places without copying code.

- **Scalability Issues:**

Imagine having hundreds of users and multiple notification types. You'd end up cluttering this class with all the notification logic.

- **Violation of Single Responsibility Principle (SRP):** The class is doing two things: handling video uploads and managing user notifications. Ideally, each class should have one responsibility.

Let us now understand how we can solve this problem using the **Observer Pattern**.

The Solution

Let's now refactor our system using the **Observer Pattern**. This version ensures a clean separation of concerns and solves all the issues we discussed earlier.

```
#include <iostream>
```

```
#include <vector>
```

```
#include <string>
```

```
#include <algorithm>
```

```
using namespace std;
```

```

// =====

// Observer Interface

// =====

class Subscriber {

public:

    virtual void update(const string& videoTitle) = 0;

    virtual ~Subscriber() {}

};


// =====

// Concrete Observer: Email

// =====

class EmailSubscriber : public Subscriber {

private:

    string email;

public:

    EmailSubscriber(const string& email) {

        this->email = email;

    }

```

```

void update(const string& videoTitle) override {

    cout << "Email sent to " << email << ": New video uploaded - " <<
videoTitle << endl;

}

};

// =====

// Concrete Observer: Mobile App

// =====

class MobileAppSubscriber : public Subscriber {

private:

    string username;

public:

    MobileAppSubscriber(const string& username) {

        this->username = username;

    }

    void update(const string& videoTitle) override {

        cout << "In-app notification for " << username << ": New video - " <<
videoTitle << endl;

    }

```

```
};
```

```
// =====
```

```
// Subject Interface
```

```
// =====
```

```
class Channel {
```

```
public:
```

```
    virtual void subscribe(Subscriber* subscriber) = 0;
```

```
    virtual void unsubscribe(Subscriber* subscriber) = 0;
```

```
    virtual void notifySubscribers(const string& videoTitle) = 0;
```

```
    virtual ~Channel() {}
```

```
};
```

```
// =====
```

```
// Concrete Subject: YouTubeChannel
```

```
// =====
```

```
class YouTubeChannel : public Channel {
```

```
private:
```

```
    vector<Subscriber*> subscribers;
```

```
    string channelName;
```

public:

```
YouTubeChannel(const string& channelName) {  
    this->channelName = channelName;  
}
```

```
void subscribe(Subscriber* subscriber) override {  
    subscribers.push_back(subscriber);  
}
```

```
void unsubscribe(Subscriber* subscriber) override {  
    subscribers.erase(std::remove(subscribers.begin(),  
subscribers.end(), subscriber), subscribers.end());  
}
```

```
void notifySubscribers(const string& videoTitle) override {  
    for (Subscriber* subscriber : subscribers) {  
        subscriber->update(videoTitle);  
    }  
}
```

```
// Simulates video upload and triggers notifications  
void uploadVideo(const string& videoTitle) {
```

```

        cout << channelName << " uploaded: " << videoTitle << "\n" << endl;

        notifySubscribers(videoTitle);

    }

};


// =====

// Client Code

// =====

int main() {

    YouTubeChannel tuf("takeUforward");


    // Add subscribers

    MobileAppSubscriber* raj = new MobileAppSubscriber("raj");

    EmailSubscriber* rahul = new EmailSubscriber("rahul@example.com");


    tuf.subscribe(raj);

    tuf.subscribe(rahul);


    // Upload video and notify all observers

    tuf.uploadVideo("observer-pattern");

```

```
// Cleanup

delete raj;

delete rahul;


return 0;
}


import java.util.*;


// =====

// Observer Interface

// =====

interface Subscriber {

    void update(String videoTitle);

}


// =====

// Concrete Observer: Email

// =====

class EmailSubscriber implements Subscriber {

    private String email;
```

```

public EmailSubscriber(String email) {

    this.email = email;

}

@Override

public void update(String videoTitle) {

    System.out.println("Email sent to " + email + ": New video uploaded - "
+ videoTitle);

}

}

// =====

// Concrete Observer: Mobile App

// =====

class MobileAppSubscriber implements Subscriber {

    private String username;

    public MobileAppSubscriber(String username) {

        this.username = username;

    }

```

```

@Override

public void update(String videoTitle) {

    System.out.println("In-app notification for " + username + ": New video
- " + videoTitle);

}

}

// =====

// Subject Interface

// =====

interface Channel {

    void subscribe(Subscriber subscriber);

    void unsubscribe(Subscriber subscriber);

    void notifySubscribers(String videoTitle);

}

// =====

// Concrete Subject: YouTubeChannel

// =====

class YouTubeChannel implements Channel {

    private List<Subscriber> subscribers = new ArrayList<>();

    private String channelName;

```

```
public YouTubeChannel(String channelName) {  
    this.channelName = channelName;  
}
```

@Override

```
public void subscribe(Subscriber subscriber) {  
    subscribers.add(subscriber);  
}
```

@Override

```
public void unsubscribe(Subscriber subscriber) {  
    subscribers.remove(subscriber);  
}
```

@Override

```
public void notifySubscribers(String videoTitle) {  
    for (Subscriber subscriber : subscribers) {  
        subscriber.update(videoTitle);  
    }  
}
```

```

// Simulates video upload and triggers notifications
public void uploadVideo(String videoTitle) {

    System.out.println(channelName + " uploaded: " + videoTitle + "\n");

    notifySubscribers(videoTitle);

}

}

// =====

// Client Code

// =====

class Main {

    public static void main(String[] args) {

        YouTubeChannel tuf = new YouTubeChannel("takeUforward");

        // Add subscribers

        tuf.subscribe(new MobileAppSubscriber("raj"));

        tuf.subscribe(new EmailSubscriber("rahul@example.com"));

        // Upload video and notify all observers

        tuf.uploadVideo("observer-pattern");

```

```
}  
  
}
```

```
from abc import ABC, abstractmethod
```

```
# =====
```

```
# Observer Interface
```

```
# =====
```

```
class Subscriber(ABC):
```

```
    @abstractmethod
```

```
    def update(self, video_title):
```

```
        pass
```

```
# =====
```

```
# Concrete Observer: Email
```

```
# =====
```

```
class EmailSubscriber(Subscriber):
```

```
    def __init__(self, email):
```

```
        self.email = email
```

```
    def update(self, video_title):
```

```
print(f"Email sent to {self.email}: New video uploaded - {video_title}")
```

```
# =====
```

```
# Concrete Observer: Mobile App
```

```
# =====
```

```
class MobileAppSubscriber(Subscriber):
```

```
    def __init__(self, username):
```

```
        self.username = username
```

```
    def update(self, video_title):
```

```
        print(f"In-app notification for {self.username}: New video -  
{video_title}")
```

```
# =====
```

```
# Subject Interface
```

```
# =====
```

```
class Channel(ABC):
```

```
    @abstractmethod
```

```
    def subscribe(self, subscriber):
```

```
        pass
```

```
    @abstractmethod
```

```
def unsubscribe(self, subscriber):
```

```
    pass
```

```
@abstractmethod
```

```
def notify_subscribers(self, video_title):
```

```
    pass
```

```
# =====
```

```
# Concrete Subject: YouTubeChannel
```

```
# =====
```

```
class YouTubeChannel(Channel):
```

```
    def __init__(self, channel_name):
```

```
        self.channel_name = channel_name
```

```
        self.subscribers = []
```

```
    def subscribe(self, subscriber):
```

```
        self.subscribers.append(subscriber)
```

```
    def unsubscribe(self, subscriber):
```

```
        self.subscribers.remove(subscriber)
```

```

def notify_subscribers(self, video_title):

    for subscriber in self.subscribers:

        subscriber.update(video_title)


# Simulates video upload and triggers notifications
def upload_video(self, video_title):

    print(f'{self.channel_name} uploaded: {video_title} \n')

    self.notify_subscribers(video_title)


# =====

# Client Code

# =====

def main():

    tuf = YouTubeChannel("takeUforward")


    # Add subscribers

    tuf.subscribe(MobileAppSubscriber("raj"))

    tuf.subscribe(EmailSubscriber("rahul@example.com"))


    # Upload video and notify all observers

    tuf.upload_video("observer-pattern")

```

```
if __name__ == "__main__":  
    main()
```

How This Solves the Problem:

Problem in Old Approach	How Observer Pattern Solves It
Channel is tightly coupled with notification logic	Each subscriber handles its own notification via update()
Not extensible for new notification types	Add new subscriber classes without modifying existing code
No reusability of logic	Notification logic is encapsulated in reusable subscriber classes
SRP Violation (upload + notify in one class)	Upload logic stays in YouTubeChannel ; notification logic is external
Difficult to manage large number of subscribers	subscribe() and unsubscribe() methods handle this cleanly

Use Cases and Limitations

Recommended Scenarios for Applying the Observer Pattern

- **State Change Propagation:**
When a change in one object must be immediately reflected across multiple dependent objects, the Observer Pattern provides a clean way to propagate this change without direct coupling.
 - **Decoupling Between Core Components:** In systems where the subject (publisher) should remain agnostic of how many observers exist or what actions they perform, the Observer Pattern promotes separation of concerns. This makes the system easier to extend and maintain.
 - **Dynamic Subscriptions at Runtime:** Situations that involve modules being added or removed dynamically (e.g., plugins, UI listeners, notification modules) benefit from the Observer Pattern, as it allows flexible attachment and detachment of observers without affecting the subject.
-

Situations Where the Observer Pattern May Fall Short

- **Excessive Observer Load**
In high-scale systems with millions of observers (e.g., when a celebrity with 10M followers goes live), a direct notification loop becomes inefficient. Such cases are better handled using **event queues**, **pub-sub architectures**, or **broadcast systems** optimized for massive concurrency.
- **Strict Control Over Notification Timing**
In environments where the timing of notifications must be tightly managed—such as financial systems or real-time analytics, deterministic control is critical. The Observer Pattern lacks fine-grained scheduling control. Systems like **message brokers** (e.g., Kafka, RabbitMQ) are more suitable in such scenarios, providing features like buffering, retries, and ordering.

In short, **Observer Pattern** works really well with a small number of observers, but to scale, it becomes essential to move toward an event-driven architecture.

Pros and Cons

Pros

- **Promotes Loose Coupling:**
Observers and subjects are decoupled. They interact only through a common interface, which improves flexibility and modularity.
 - **Open for Extension:**
New types of observers can be added without modifying the subject class, adhering to the Open/Closed Principle.
 - **Supports Dynamic Subscription:**
Observers can be attached or detached at runtime, enabling highly configurable and adaptable systems.
 - **Encourages Reusability:**
Different observer implementations can be reused across subjects or contexts without duplication of logic.
-

Cons

- **Unpredictable Update Sequences:**
If the order of observer notifications matters, it may be hard to manage as the pattern does not guarantee update order.
- **Performance Bottlenecks at Scale:**
Notifying a large number of observers synchronously can degrade performance in high-scale systems.

- **Risk of Memory Leaks:**
Failure to unsubscribe unused observers may result in lingering references and memory issues.
 - **Difficult Debugging:**
Since interactions happen indirectly through interfaces, tracing the source of bugs or unwanted updates can be challenging.
 - **Tight Timing Coupling:**
All observers are notified immediately. Delayed or controlled delivery of events is not supported natively.
-

Real-Life Use Cases

The Observer Pattern is widely used in real-world systems that require **automatic propagation of changes** across dependent components. Here are a few notable examples:

- **UI Event Handling:**
In GUI frameworks, buttons, sliders, and input fields use observers (listeners) to respond to user actions like clicks or typing.
- **News or Blog Subscriptions:**
Readers subscribe to news feeds or blog updates. When new content is published, all subscribers are notified instantly.
- **Stock Market Tickers:**
Trading platforms subscribe to stock price changes. Whenever prices update, relevant modules (charts, alerts, watchlists) are notified in real-time.
- **File System Watchers:**
IDEs or OS-level watchers use observers to track file changes. Once a file is modified, all registered tools or services (like compilers or sync tools) are triggered.
- **Social Media Notifications:**
Platforms like YouTube or Instagram notify followers when someone they follow posts new content.

Class Diagram

TUF ➔

